As `WebApplication` automatically adds `EndpointMiddleware` to the *end* of the middleware pipeline, the `WelcomePageMiddleware` will *always* be ahead of it, so it always generates a response before the endpoint can execute in this example.

TIP You should always consider the order of middleware when adding it to `WebApplication`. Middleware added earlier in the pipeline will run (and potentially return a response) before middleware added later.

All the examples shown so far try to handle an incoming request and generate a response, but it's important to remember that the middleware pipeline is bidirectional. Each middleware component gets an opportunity to handle both the incoming request and the outgoing response. The order of middleware is most important for those components that create or modify the outgoing response.

In listing 4.3, I included `DeveloperExceptionPageMiddleware` at the start of the application's middleware pipeline, but it didn't seem to do anything. Error-handling middleware characteristically ignores the incoming request as it arrives in the pipeline; instead, it inspects the outgoing response, modifying it only when an error has occurred. In the next section, I discuss the types of error-handling middleware that are available to use with your application and when to use them.

4.3 Handling errors using middleware

Errors are a fact of life when you're developing applications. Even if you write perfect code, as soon as you release and deploy your application, users will find a way to break it, by accident or intentionally! The important thing